task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py

```python
import os
import time
import numpy as np
from concurrent.futures import ProcessPoolExecutor
STUDENT_ID = "230103227"
ARRAY_SIZES = [
    100,
    1_000,
    10_000,
    100_000,
    1_000_000,
    10_000_000
]
A = 2.5
B = 1.5
REPEATS = 3
def calculate_chunk(args):
    x_chunk, a, b = args
    return a * x_chunk + b
def serial_operation(x, a, b):
    return a * x + b
def parallel_operation(x, a, b, executor, workers):
    chunks = np.array_split(x, workers)
    args = [(chunk, a, b) for chunk in chunks if len(chunk) > 0]
    results = list(executor.map(calculate_chunk, args))
    return np.concatenate(results)
def measure_serial(x):
    times = []

    for _ in range(REPEATS):
```

task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py

```python
def measure_serial(x):

    for _ in range(REPEATS):
        start = time.perf_counter()
        y = serial_operation(x, A, B)
        elapsed = time.perf_counter() - start
        times.append(elapsed)
    return min(times), y
def measure_parallel(x, executor, workers):
    times = []
    for _ in range(REPEATS):
        start = time.perf_counter()
        y = parallel_operation(x, A, B, executor, workers)
        elapsed = time.perf_counter() - start
        times.append(elapsed)
    return min(times), y
def main():
    workers = os.cpu_count() or 1
    rng = np.random.default_rng(int(STUDENT_ID[-3:]))
    print("TASK 5: THE OVERHEAD CROSSOVER POINT")
    print(f"Student ID: {STUDENT_ID}")
    print(f"Logical CPU cores: {workers}")
    print(f"Parallel workers: {workers}")
    print(f"Repeats per test: {REPEATS}")
    print()
    results = []
    with ProcessPoolExecutor(max_workers=workers) as executor:
        for n in ARRAY_SIZES:
            x = rng.random(n, dtype=np.float64)
```

task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py

```
    def main():
        with ProcessPoolExecutor(max_workers=workers) as executor:

                serial_time, y_serial = measure_serial(x)
                parallel_time, y_parallel = measure_parallel(
                    x, executor, workers
                )
                if not np.allclose(y_serial, y_parallel):
                    print(f"ERROR: Results do not match for N={n}")
                    return
                speedup = serial_time / parallel_time
                if parallel_time < serial_time:
                    winner = "Parallel faster"
                elif parallel_time > serial_time:
                    winner = "Serial faster"
                else:
                    winner = "Equal"
                results.append(
                    (n, serial_time, parallel_time, speedup, winner)
                )
        print(
            f"{'N':<12}"
            f"{'Serial (ms)':<16}"
            f"{'Parallel (ms)':<18}"
            f"{'Speedup':<12}"
            f"{'Result'}"
        )
        print("-" * 75)

        for n, serial_time, parallel_time, speedup, winner in results:
```

task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py

```
    def main():

        for n, serial_time, parallel_time, speedup, winner in results:
            print(
                f"{n:<12}"
                f"{serial_time * 1000:<16.3f}"
                f"{parallel_time * 1000:<18.3f}"
                f"{speedup:<12.3f}"
                f"{winner}"
            )
        print()
        print("CROSSOVER ANALYSIS")
        slower_sizes = [
            n for n, serial, parallel, _, _ in results
            if parallel > serial
        ]
        faster_sizes = [
            n for n, serial, parallel, _, _ in results
            if parallel < serial
        ]
        if not faster_sizes:
            print("No crossover found: parallel execution was never faster.")
        elif not slower_sizes:
            print("Parallel execution was faster for all tested sizes.")
        else:
            last_slower = max(slower_sizes)
            first_faster = min(faster_sizes)
            if last_slower < first_faster:
                print(
                    f"Observed crossover range: N > {last_slower} "
```

task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py

```
def main():
    else:
        last_slower = max(slower_sizes)
        first_faster = min(faster_sizes)
        if last_slower < first_faster:
            print(
                f"Observed crossover range: N > {last_slower} "
                f"and N <= {first_faster}."
            )
            print(
                f"Parallel first becomes faster among tested sizes "
                f"at N = {first_faster}."
            )
        else:
            print(
                "Performance is not a simple one-way transition "
                "for the tested sizes."
            )
            print(f"Sizes where serial was faster: {slower_sizes}")
            print(f"Sizes where parallel was faster: {faster_sizes}")

    print()
    print("Note: The crossover is measured only for the tested sizes.")
    print("It may vary with system load and hardware.")


if __name__ == "__main__":
    main()
```

Run | task5

```
C:\Users\Акерке\.local\bin\uv.exe run C:/Users/Акерке/PycharmProjects/PythonP
TASK 5: THE OVERHEAD CROSSOVER POINT
Student ID: 230103227
Logical CPU cores: 4
Parallel workers: 4
Repeats per test: 3

N           Serial (ms)     Parallel (ms)     Speedup     Result
--------------------------------------------------------------------------------
100         0.006           1.524             0.004       Serial faster
1000        0.006           1.244             0.005       Serial faster
10000       0.010           1.350             0.007       Serial faster
100000      0.712           3.819             0.186       Serial faster
1000000     6.454           31.018            0.208       Serial faster
10000000    60.676          311.603           0.195       Serial faster

CROSSOVER ANALYSIS
No crossover found: parallel execution was never faster.

Note: The crossover is measured only for the tested sizes.
It may vary with system load and hardware.

Process finished with exit code 0
```

task1.py | task1_output.txt | task2.py | task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py | task5_output.txt

```
TASK 5: THE OVERHEAD CROSSOVER POINT
Student ID: 230103227
Logical CPU cores: 4
Parallel workers: 4
Repeats per test: 3

N           Serial (ms)     Parallel (ms)     Speedup     Result
--------------------------------------------------------------------------------
100         0.006           1.524             0.004       Serial faster
1000        0.006           1.244             0.005       Serial faster
10000       0.010           1.350             0.007       Serial faster
100000      0.712           3.819             0.186       Serial faster
1000000     6.454           31.018            0.208       Serial faster
10000000    60.676          311.603           0.195       Serial faster

CROSSOVER ANALYSIS
No crossover found: parallel execution was never faster.

Note: The crossover is measured only for the tested sizes.
It may vary with system load and hardware.
```